

多项式部分

```
#define N 3000050
#define mod 998244353
const ld PI=acos(-1.0);
using std::string;
string s1,s2;
int read(){int p=0,flg=1;char c=getchar();while(c<'0' || c>'9'){if(c=='-') flg=-1;c=getchar();}wh:
void write(int x){if (!x)return;write(x/10);putchar(x%10+'0');}
namespace Poly {
    int p[N],q[N],r[N],w[N],invnum[N];
    int add(int x,int y){return x+y>mod?x+y-mod:x+y;}
    int dec(int x,int y){return x-y<0?x-y+mod:x-y;}
    int ext(int n){return n==1?1:(1<<(std::__lg(n-1)+1));}
    int qpow(int a,int b){int res=1;for (;b;b>>=1,a=1ll*a*a%mod)if (b&1)res=1ll*res*a%mod;return
    void get_r(int n){for (int i=0;i<n;i++)r[i]=(r[i>>1]>>1)|(i&1?n>>1:0);}
    void print(int *a,int n){for (int i=0;i<n;i++)printf("%d%c",a[i], " \n"[i==n-1]);}
    void init_poly(){
        for(int k=2,m=1;k<=N;k<=1,m<=1){
            w[m]=1;
            for(int i=m+1,x=qpow(3,(mod-1)/k);i<k;i++) w[i]=1ll*w[i-1]*x%mod;
        }invnum[1]=1;
        for (int i=2;i<N;i++)invnum[i]=1ll*(mod-mod/i)*invnum[mod%i]%mod;
    }
    void NTT(int *a,int n,int op){
        for (int i=0;i<n;i++)if (i<r[i])std::swap(a[i],a[r[i]]);
        for (int k=2,m=1;k<=n;m<=1,k<=1){
            for (int i=0;i<n;i+=k)
                for (int j=i,*x=w+m;j<i+m;x++,j++){
                    int v=1ll*a[j+m]*(x%mod);
                    a[j+m]=dec(a[j],v);a[j]=add(a[j],v);
                }
        }
        if (op==-1){
            std::reverse(a+1,a+n);
            for (int i=0,v=qpow(n,mod-2);i<n;i++)a[i]=1ll*a[i]*v%mod;
        }
    }
    void mul(int *a,int *b,int n,int m){
        if (!n||!m)return;
```

```

    int len=ext(n+m-1);
    for (int i=0;i<len;i++)p[i]=i<n?a[i]:0,q[i]=i<m?b[i]:0;
    get_r(len);
    NTT(p,len,1);NTT(q,len,1);
    for (int i=0;i<len;i++)a[i]=111*p[i]*q[i]%mod;
    NTT(a,len,-1);
}

void _inv(int *a,int *b,int n){
    static int c[N],d[N];
    n=ext(n);memset(b,0,4*n);b[0]=qpow(a[0],mod-2);
    for (int k=2;k<=n;k<=1){
        for (int i=0;i<k<1;i++)c[i]=i<k?a[i]:0,d[i]=i<k>>1?b[i]:0;
        get_r(k);NTT(d,k,1);
        for (int i=0;i<k;i++)d[i]=111*d[i]*d[i]%mod;
        NTT(d,k,-1);get_r(k<1);NTT(c,k<1,1);NTT(d,k<1,1);
        for (int i=0;i<k<1;i++)c[i]=111*c[i]*d[i]%mod;
        NTT(c,k<1,-1);
        for (int i=0;i<k;i++)b[i]=(211*b[i]-c[i]+mod)%mod;
    }
}

void _diff(int *a,int *b,int n){//a-n个数
    for (int i=1;i<n;i++)b[i-1]=111*i*a[i]%mod;b[n-1]=0;
}

void _integ(int *a,int *b,int n){//b-n个数
    for (int i=1;i<n;i++)b[i]=111*invnum[i]*a[i-1]%mod;b[0]=0;
}

void _ln(int *a,int *b,int n){
    static int c[N],d[N];
    assert(a[0]==1);
    n=ext(n);
    _inv(a,c,n),_diff(a,d,n),mul(c,d,n,n);_integ(c,b,n);
}

void _exp(int *a,int *b,int n){
    static int c[N];
    assert(a[0]==0);
    n=ext(n);memset(b,0,4*n);memset(c,0,4*n);b[0]=1;
    for (int k=2;k<=n;k<=1){
        _ln(b,c,k);
        for (int i=0;i<k;i++)c[i]=dec(a[i],c[i]);
        c[0]++;mul(b,c,k,k);
    }
}

```

```

void _sqrt(int *a,int *b,int n){
    static int c[N],d[N];
    static const int inv2=(mod+1)>>1;
    assert(a[0]==1);
    n=ext(n);memset(b,0,4*n);b[0]=1;
    for (int k=2;k<=n;k<=1){
        memcpy(c,a,4*k);_inv(b,d,k),mul(c,d,k,k);
        for (int i=0;i<k;i++)b[i]=111*(b[i]+c[i])*inv2%mod;
    }
}

void _pow(int *a,int *b,int n,string k){
    static int c[N],d[N];
    int u=0,k1=0,k2=0;
    memset(b,0,4*n);
    for (int i=0;i<k.size();i++)k1=(1011*k1+k[i]-'0')%mod,k2=(1011*k2+k[i]-'0')%(mod-1);
    for (u=0;u<n&&!a[u];u++);
    if ((u&&k.size())>=5||111*u*k1>=n)return;
    for (int i=u,x=qpow(a[u],mod-2);i<n;i++)c[i-u]=111*a[i]*x%mod;
    _ln(c,d,n-u);
    for (int i=0;i<n-u;i++)d[i]=111*d[i]*k1%mod;
    _exp(d,c,n-u);
    for (int i=u*k1,x=qpow(a[u],k2);i<n;i++)b[i]=111*c[i-u*k1]*x%mod;
}

void _pow_p(int *a,int *b,int n,string k){
    static int c[N],d[N];
    int u=0,k1=0,k2=0;
    for (int i=0;i<k.size();i++)k1=(1011*k1+k[i]-'0')%mod;
    _ln(a,d,n);
    for (int i=0;i<n;i++)d[i]=111*d[i]*k1%mod;
    _exp(d,b,n);
}

void _div(int *a,int *b,int *q,int *r,int n,int m){
    static int c[N],d[N],e[N];
    int len=n-m+1;
    for (int i=0;i<len;i++)c[i]=a[n-1-i];
    for (int i=0;i<len;i++)d[i]=i<m?b[m-1-i]:0;
    _inv(d,e,len);mul(c,e,len,len),std::reverse(c,c+len);
    memcpy(q,c,4*len),mul(c,b,len,m);
    for (int i=0;i<m;i++)r[i]=dec(a[i],c[i]);
}

}

using namespace Poly;

```

数论部分

//埃筛

```
int vis[N],pri[N],cnt;
void Eratosthenes(int n){
    vis[0]=vis[1]=0;cnt=0;
    for(int i=2;i<=n;i++)vis[i]=1;
    for (int i=2;i<=n;i++){
        if (vis[i]){
            pri[cnt++]=i;
            if (1ll*i*i>n)continue;
            for (int j=i*i;j<=n;j+=i){
                vis[j]=0;
            }
        }
    }
}
```

//欧拉筛

```
void Euler(int n){
    cnt=0;
    for (int i=2;i<=n;i++)vis[i]=0;
    for (int i=2;i<=n;i++){
        if (!vis[i])pri[cnt++]=i;
        for (int j=1;j<=cnt;j++){
            if (1ll*i*pri[j]>n)break;
            vis[i*pri[j]]=1;
            if (i%pri[j]==0)break;
        }
    }
}
```

//phi

```
int phi[N];
void get_phi(int n){
    cnt=0;phi[1]=1;
    for (int i=2;i<=n;i++)vis[i]=0;
    for (int i=2;i<=n;i++){
        if (!vis[i])pri[++cnt]=i,phi[i]=i-1;
        for (int j=1;j<=cnt;j++){
            if (1ll*i*pri[j]>n)break;
            if (i%pri[j]==0){
                vis[i*pri[j]]=1;phi[i*pri[j]]=pri[j]*phi[i];
                break;
            }
        }
    }
}
```

```

        }
        vis[i*pri[j]]=1;phi[i*pri[j]]=(pri[j]-1)*phi[i];
    }
}
//mu
int mu[N];
void get_mu(int n){
    cnt=0;mu[1]=1;
    for (int i=2;i<=n;i++)vis[i]=0;
    for (int i=2;i<=n;i++){
        if (!vis[i])pri[++cnt]=i,mu[i]=-1;
        for (int j=1;j<=cnt;j++){
            if (1ll*i*pri[j]>n)break;
            if (i%pri[j]==0){
                vis[i*pri[j]]=1;mu[i*pri[j]]=0;
                break;
            }
            vis[i*pri[j]]=1;mu[i*pri[j]]=-mu[i];
        }
    }
}
//约数个数
int d[N],num[N];
void get_d(int n){
    cnt=0;d[1]=1;
    for (int i=2;i<=n;i++)vis[i]=0;
    for (int i=2;i<=n;i++){
        if (!vis[i])pri[++cnt]=i,d[i]=2,num[i]=1;
        for (int j=1;j<=cnt;j++){
            if (1ll*i*pri[j]>n)break;
            if (i%pri[j]==0){
                vis[i*pri[j]]=1;num[i*pri[j]]=num[i]+1;d[i*pri[j]]=d[i]*(num[i*pri[j]]+1)/num[i];
                break;
            }
            vis[i*pri[j]]=1;num[i*pri[j]]=1;d[i*pri[j]]=2*d[i];
        }
    }
}
//约数和
int dsum[N],gsum[N];
void get_dsum(int n){
    cnt=0;dsum[1]=1;

```

```

for (int i=2;i<=n;i++)vis[i]=0;
for (int i=2;i<=n;i++){
    if (!vis[i])pri[++cnt]=i,dsum[i]=1+i,gsum[i]=1+i;
    for (int j=1;j<=cnt;j++){
        if (1ll*i*pri[j]>n)break;
        if (i%pri[j]==0){
            vis[i*pri[j]]=1;
            gsum[i*pri[j]]=pri[j]*gsum[i]+1;
            dsum[i*pri[j]]=dsum[i]*(gsum[i*pri[j]])/gsum[i];
            break;
        }
        vis[i*pri[j]]=1;
        gsum[i*pri[j]]=1+pri[j];
        dsum[i*pri[j]]=(1+pri[j])*dsum[i];
    }
}
}
//狄利克雷卷积
for(int i = 1; i <= tot; i++) {
    for(int j = n / pri[i]; j >= 1; j--) {
        int t = pri[i];
        for(long long k = j * pri[i]; k <= n; k *= pri[i]) {
            f[k] += f[j] * g[t];
            t *= pri[i];
        }
    }
}
void Dir(){//g积性
    for (int i=1;i<=cnt;i++){
        for (int j=n/pri[i];j>=1;j--){
            ll cnt=1,p=pri[i];
            for (ll k=j*p;k<=1ll*n;k*=p,cnt++){
                a[k]=add(a[k],1ll*a[j]*g[cnt]%mod);
            }
        }
    }
}
int exgcd(int a,int b,int& x,int& y){
    if (!b)return x=1,y=0,a;
    int d=exgcd(b,a%b,x,y);
    int t=x;
    x=y;
    y=t-(a/b)*y;
}

```

```

    return d;
}
int a[N],r[N];
int crt(int k){
    int n=1,ans=0;
    for (int i=1;i<=k;i++)n*=r[i];
    for (int i=1;i<=k;i++){
        int m=n/r[i];
        ans=(ans+a[i]*m*pow(m,r[i])%n)%n;
    }
    return (ans%n+n)%n;
}
ll BSGS(ll a,ll b,ll p){
    mp.clear();
    if (b==1)return 0;
    a%=p;b%=p;ll m=ceil(sqrt(p)),res=1;
    for (int i=0;i<m;i++){
        if (!mp.count(res))mp[res]=i;
        res=(res*a)%p;
    }
    ll A=inv(res,p);
    res=b;
    for (int i=0;i<m;i++){
        if (mp.count(res))return i*m+mp[res];
        res=res*A%p;
    }
    return -1;
}
ll Lucas(ll n,ll m){
    if(!m)return 1;
    return C(n%p,m%p)*Lucas(n/p,m/p)%p;
}

```

线性代数部分

//线性基

```
bool insert(int x){
    for (int i=30;i>=0;i--){
        if (x>>i&1){
            if (!val[i]){
                val[i]=x;cnt++;
                return 1;
            }
            x^=val[i];
        }
    }
    return 0;
}
```

//第k小

```
int qry_kth(int k){
    int ans=0,ct=1<<cnt;
    for (int i=30;i>=0;i--){
        if (val[i]){
            if (k>(ct>>=1)){
                k-=ct;
                if (~ans>>i&1)ans^=val[i];
            }
            else if (ans>>i&1)ans^=val[i];
        }
    }
    return ans;
}
```

//求排名

```
int qry_rk(int x){
    int ans=0,ct=(1<<cnt),pre=0;x++;
    for (int i=30;i>=0;i--){
        if (val[i]){
            ans+=(x>>i&1)*(ct>>=1);
            if ((x^pre)>>i&1)pre^=val[i];
        }
        else if ((x^pre)>>i&1){
            if (x>>i&1)ans+=ct;
            break;
        }
    }
}
```



```
return ans;
```

```
}
```

图论部分

```
//数剖
struct edge{
    int v,nex;
}e[N<<1];
void add(int u,int v){
    e[++cnt]={v,head[u]};head[u]=cnt;
}
struct segtree{
#define ls (p<<1)
#define rs (p<<1|1)
#define mid ((l+r)>>1)
    int t[N<<2],tag[N<<2];
    void pushup(int p){
        t[p]=t[ls]+t[rs];t[p]%=mod;
    }
    void pushdown(int p,int l,int r){
        if(!tag[p])return;
        t[ls]+=tag[p]*(mid-l+1);t[ls]%=mod;
        t[rs]+=tag[p]*(r-mid);t[rs]%=mod;
        tag[ls]+=tag[p];tag[ls]%=mod;
        tag[rs]+=tag[p];tag[rs]%=mod;
        tag[p]=0;
    }
    void change(int p,int l,int r,int x,int y,int k){
        if(x>r||l>y)return;
        if(l>=x&&r<=y){
            t[p]+=(r-l+1)*k;tag[p]+=k;t[p]%=mod;tag[p]%=mod;
            return;
        }
        pushdown(p,l,r);
        change(ls,l,mid,x,y,k);
        change(rs,mid+1,r,x,y,k);
        pushup(p);
    }
    int qry(int p,int l,int r,int x,int y){
        if(x>r||l>y)return 0;
        if(l>=x&&r<=y)return t[p];
        pushdown(p,l,r);
        return (qry(ls,l,mid,x,y)+qry(rs,mid+1,r,x,y))%mod;
    }
}
```

```

}T;
void dfs1(int u,int f){
    siz[u]=1;fa[u]=f;dep[u]=dep[fa[u]]+1;
    for(int i=head[u];i;i=e[i].nex){
        int v=e[i].v;
        if(v==f)continue;
        dfs1(v,u);
        siz[u]+=siz[v];
        if(siz[v]>siz[son[u]])son[u]=v;
    }
}
void dfs2(int u,int topp){
    dfn[u]++;dfnct;
    top[u]=topp;
    T.change(1,1,n,dfn[u],dfn[u],w[u]);
    if(!son[u])return;
    dfs2(son[u],topp);
    for(int i=head[u];i;i=e[i].nex){
        int v=e[i].v;
        if(v==fa[u]||v==son[u])continue;
        dfs2(v,v);
    }
}
void upd_path(int u,int v,int k){
    while(top[u]!=top[v]){
        if(dep[top[u]]<dep[top[v]])swap(u,v);
        T.change(1,1,n,dfn[top[u]],dfn[u],k);
        u=fa[top[u]];
    }
    if(dep[u]<dep[v])swap(u,v);
    T.change(1,1,n,dfn[v],dfn[u],k);
}
ll qry_path(int u,int v){
    ll res=0;
    while(top[u]!=top[v]){
        if(dep[top[u]]<dep[top[v]])swap(u,v);
        res+=1ll*T.qry(1,1,n,dfn[top[u]],dfn[u]);res%=mod;
        u=fa[top[u]];
    }
    if(dep[u]<dep[v])swap(u,v);
    res+=1ll*T.qry(1,1,n,dfn[v],dfn[u]);
    return res%mod;
}

```

```

void upd_tree(int u,int k){
    T.change(1,1,n,dfn[u],dfn[u]+siz[u]-1,k);
}
ll qry_tree(int u){
    return 1ll*T.qry(1,1,n,dfn[u],dfn[u]+siz[u]-1)%mod;
}
//二分图最大匹配
bool dfs(int u,int falg){
    if(vis[u]==falg) return 0;
    vis[u]=falg;
    for(int i=0;i<g[u].size();i++){
        int v=g[u][i];
        if(!mat[v]||dfs(mat[v],falg)){
            mat[v]=u;
            return 1;
        }
    }
    return 0;
}
//缩点
vector<int>g[N],g2[N];
int dfn[N],low[N],w[N],tot,cnt,top,dc,col;
int sta[N],n,m,head[N],vis[N];
int ww[N],ct,a[N];
int in[N],p[N],pct,dp[N],ans;
struct edge{
    int u,v,nex;
}e[N<<1];
void add(int u,int v){
    e[++cnt]={u,v,head[u]};head[u]=cnt;
}
void dfs(int u){
    sta[++top]=u;vis[u]=1;dfn[u]=low[u]=++dc;
    for(int i=head[u];i;i=e[i].nex){
        int v=e[i].v;
        if(!dfn[v]){
            dfs(v);
            low[u]=min(low[u],low[v]);
        }
        else if(vis[v])low[u]=min(low[u],dfn[v]);
    }
    if(low[u]==dfn[u]){
        col++;
    }
}

```

```

        while(1){
            a[sta[top]]=col;
            ww[col]+=w[sta[top]];
            vis[sta[top]]=0;top--;
            if(sta[top+1]==u)break;
        }
    }
}

std::queue<int>q;
void topu(){
    for(int i=1;i<=col;i++)
        if(!in[i])q.push(i);
    while(!q.empty()){
        int u=q.front();q.pop();
        p[++pct]=u;
        for(int i=0;i<g[u].size();i++){
            int v=g[u][i];
            if(--in[v]==0)q.push(v);
        }
    }
}

//2sat
int head[N],cnt,n,m;
struct edge{
    int v,nex;
}e[N];
void add(int u,int v){
    e[++cnt]={v,head[u]};
    head[u]=cnt;
}

int dfn[N],low[N],dc,sc,sta[N],vis[N],a[N],col;
void tarjan(int x){
    dfn[x]=low[x]=++dc;
    sta[++sc]=x;vis[x]=1;
    for(int i=head[x];i;i=e[i].nex){
        int v=e[i].v;
        if(!dfn[v]){
            tarjan(v);
            low[x]=std::min(low[x],low[v]);
        }
        else if(vis[v])
            low[x]=std::min(low[x],dfn[v]);
    }
}

```

```

        if(dfn[x]==low[x]){
            col++;
            while(1){
                a[sta[sc]]=col;
                vis[sta[sc]]=0;
                if(x==sta[sc--])break;
            }
        }
    }
}

int main(){
    scanf("%d%d",&n,&m);
    for(int i=1,a,aa,b,bb;i<=m;i++){
        scanf("%d%d%d%d",&a,&aa,&b,&bb);
        add(a+n*(aa&1),b+n*(bb&1));
        add(b+n*(bb&1),a+n*(aa&1));
    }
    for(int i=1;i<=n<<1;i++)
        if(!dfn[i])tarjan(i);
    for(int i=1;i<=n;i++){
        if(a[i]==a[i+n]){
            puts("IMPOSSIBLE");
            return 0;
        }
    }
    puts("POSSIBLE");
    for(int i=1;i<=n;i++)
        printf("%d ",a[i]<a[i+n]);
    return 0;
}

//割点
struct edge{
    int v,nex;
}e[N<<1];
void add(int u,int v){
    e[++cnt]={v,head[u]};head[u]=cnt;
}

void dfs(int u,int f){
    int child=0;dfn[u]=low[u]=++dc;
    for(int i=head[u];i;i=e[i].nex){
        int v=e[i].v;
        if(v==f)continue;
        if(!dfn[v]){
            dfs(v,u);
            low[u]=min(low[u],low[v]);

```

```

        if(low[v]>=dfn[u]&&u!=f)cut[u]=1;
        if(u==f)child++;
    }
    low[u]=min(low[u],dfn[v]);
}
if(u==f&&child>=2)cut[u]=1;
}
//网络流
struct Dinic{
    struct edge{
        int from,to,cap,flow;
    };
    int n,m,s,t;
    int num,vis[N],d[N],cur[N];
    vector<int>G[N];
    vector<edge>E;
    void init(int n){
        E.clear();
        for(int i=0;i<=n;i++)G[i].clear();
    }
    void add(int from,int to,int cap){
        E.push_back({from,to,cap,0});
        E.push_back({to,from,0,0});
        m=E.size();
        G[from].push_back(m-2);
        G[to].push_back(m-1);
    }
    bool BFS(){
        std::queue<int>q;
        q.push(s);
        d[s]=0;vis[s]=++num;
        while(!q.empty()){
            int x=q.front();q.pop();
            for(int i=0;i<G[x].size();i++){
                edge e=E[G[x][i]];
                if(vis[e.to]!=num&&e.cap>e.flow){
                    vis[e.to]=num;
                    d[e.to]=d[x]+1;
                    q.push(e.to);
                }
            }
        }
    }
    return vis[t]==num;
}

```

```

    }
    int DFS(int x,int a){
        if(x==t||a==0)return a;
        int f,flow=0;
        for(int&i=cur[x];i<G[x].size();i++){
            edge&e=E[G[x][i]];
            if(d[x]+1==d[e.to]&&(f=DFS(e.to,std::min(a,e.cap-e.flow)))>0){
                flow+=f;
                e.flow-=f;
                E[G[x][i]^1].flow+=f;
                a-=f;
                if(a==0)break;
            }
        }
        return flow;
    }
}

int Manflow(int s,int t){
    this->s=s;
    this->t=t;
    int flow=0;
    while(BFS()){
        memset(cur,0,sizeof(cur));
        flow+=DFS(s,INF);
    }
    return flow;
}

}T;
//点双
int n,m;
int ct[M<<1],head[N],cnt,dfn[N],low[N],dc;
struct edge{
    int v,nex;
}e[M<<1];
void add(int u,int v){
    e[++cnt]={v,head[u]};head[u]=cnt;
}
int sta[N],top;
vector<int>dcc[N];
int ans,col[N];
void tarjan(int u,int f){
    dfn[u]=low[u]=++dc;
    sta[++top]=u;
    if(head[u]==0&&f==0){

```



```

        dcc[++ans].pb(u);
    }
    for(int i=head[u];i;i=e[i].nex){
        int v=e[i].v;
        if(!dfn[v]){
            tarjan(v,u);
            low[u]=min(low[u],low[v]);
            if(low[v]>=dfn[u]){
                ans++;//注意是v，一个点可以在多个点双
                while(sta[top+1]!=v)dcc[ans].pb(sta[top--]);
                dcc[ans].pb(u);
            }
        }
        else if(v!=f)low[u]=min(low[u],low[v]);
    }
}

//边双
int n,m;
int ct[M<<1],head[N],cnt,dfn[N],low[N],dc;
struct edge{
    int v,nex;
}e[M<<1];
void add(int u,int v){
    e[++cnt]={v,head[u]};head[u]=cnt;
}
void tarjan(int u,int f){
    dfn[u]=low[u]=++dc;
    for(int i=head[u];i;i=e[i].nex){
        int v=e[i].v;
        if(!dfn[v]){
            tarjan(v,i);
            if(low[v]>dfn[u]){
                ct[i]=ct[i^1]=1;
            }
            low[u]=min(low[u],low[v]);
        }
        else if(i!=(f^1))low[u]=min(low[u],dfn[v]);
    }
}

vector<vector<int>>>dcc;
int ans,col[N];
void dfs(int u,int c){
    col[u]=c;

```

```

        dcc[c-1].push_back(u);
        for(int i=head[u];i;i=e[i].nex){
            int v=e[i].v;
            if(col[v]||ct[i])continue;
            dfs(v,c);
        }
    }
}
//最下费用最大流
int head[N],n,m,s,t,cnt=1,cur[N],dis[N],ret=0,ans;
bool vis[N];
struct edge{
    int v,f,c,nex;
}e[100100];
void add(int u,int v,int f,int c){
    e[++cnt]={v,f,c,head[u]};
    head[u]=cnt;
}
void addedge(int u,int v,int f,int c){
    add(u,v,f,c);
    add(v,u,0,-c);
}
bool spfa(){
    std::queue<int>q;
    memcpy(cur,head,sizeof(head));
    memset(dis,INF,sizeof(dis));
    q.push(s);dis[s]=0;vis[s]=1;
    while(!q.empty()){
        int x=q.front();q.pop();
        vis[x]=0;
        for(int i=head[x];i;i=e[i].nex){
            int v=e[i].v;
            if(e[i].f&&dis[v]>dis[x]+e[i].c){
                dis[v]=dis[x]+e[i].c;
                if(vis[v]!=1){
                    q.push(v);
                    vis[v]=1;
                }
            }
        }
    }
}
return dis[t]!=INF;
}
int dfs(int x,int a){

```

```

        if(x==t||!a)return a;
        int flow=0,f;
        vis[x]=1;
        for(int&i=cur[x];i&&flow<a;i=e[i].nex){
            int v=e[i].v;
            if(e[i].f&&vis[v]!=1&&dis[v]==dis[x]+e[i].c&&(f=dfs(v,min(a-flow,e[i].f)))){

                ret+=e[i].c*f;
                e[i].f-=f;
                e[i^1].f+=f;
                flow+=f;
            }
        }
        vis[x]=0;
        return flow;
    }
    int mcmf(){
        int flow=0,x;
        while(spfa()){
            while(x=dfs(s,INF))flow+=x;
        }
        return flow;
    }
    //二分图最大匹配
    bool dfs(int u,int falg){
        if(vis[u]==falg) return 0;
        vis[u]=falg;
        for(int i=0;i<g[u].size();i++){
            int v=g[u][i];
            if(!mat[v]||dfs(mat[v],falg)){
                mat[v]=u;
                return 1;
            }
        }
        return 0;
    }

    int main(){
        n=rd();m=rd();e=rd();
        for(int u,v;e;e--){
            u=rd();v=rd();
            g[u].push_back(v);
        }
    }

```

```

        for(int i=1;i<=n;i++)if(dfs(i,i))ans++;
        printf("%d",ans);
        return 0;
    }
    //二分图最大权完美匹配
    #include<bits/stdc++.h>
    #define N 1005
    #define ll long long
    #define INF 0x3f3f3f3f
    const ll inf=0x3f3f3f3f3f3f3f3f;
    ll n,m,vis[N],a[N],b[N],s[N],mat[N],x,pre[N],cur,pn;
    ll d[N][N],w;
    void BFS(int x){
        memset(vis,0,sizeof vis);
        memset(s,INF,sizeof s);
        memset(pre,0,sizeof pre);
        cur=pn=0;mat[cur]=x;
        do{
            x=mat[cur];vis[cur]=1;
            ll dis=inf;
            for(int i=1;i<=n;++i){
                if(vis[i])continue;
                ll t=a[x]+b[i]-d[x][i];
                if(t<s[i]){
                    s[i]=t;pre[i]=cur;
                }
                if(s[i]<dis){
                    dis=s[i];pn=i;
                }
            }
            for(int i=0;i<=n;i++){
                if(vis[i])a[mat[i]]-=dis,b[i]+=dis;
                else s[i]-=dis;
            }
            cur=pn;
        }while(mat[cur]);
        while(cur){
            mat[cur]=mat[pre[cur]];
            cur=pre[cur];
        }
    }
    ll km(){
        for(int i=0;i<=n;i++)a[i]=b[i]=mat[i]=0;

```

```

        for(int i=1;i<=n;i++)
            BFS(i);
    ll res=0;
    for(int i=1;i<=n;i++)res+=d[mat[i]][i];
    return res;
}

int main(){
    scanf("%lld%lld",&n,&m);
    for(int i=1;i<=n;i++)
        for(int j=1;j<=n;j++)
            d[i][j]=-inf;
    for(int u,v,i=1;i<=m;i++){
        scanf("%lld%lld%lld",&u,&v,&w);
        d[u][v]=std::max(d[u][v],1ll*w);
    }
    printf("%lld\n",km());
    for(int i=1;i<=n;i++)printf("%lld ",mat[i]);
    puts(" ");
    return 0;
}

// 点分治
#include<bits/stdc++.h>
#define N 1000006
#define INF 0x3f3f3f3f

int head[N],cnt,cntt,n,m,rt,p[N],ret[N];
struct edge{
    int v,w,nex;
}e[N];
void add(int u,int v,int w){
    e[++cnt]={v,w,head[u]};
    head[u]=cnt;
}

int sum,siz[N],maxx[N];
bool vis[N],tf[N];
void calcsiz(int u,int fa){
    maxx[u]=0;siz[u]=1;
    for(int i=head[u];i;i=e[i].nex){
        int v=e[i].v;
        if(v==fa||vis[v])continue;
        calcsiz(v,u);
        maxx[u]=std::max(maxx[u],siz[v]);
        siz[u]+=siz[v];
    }
}

```

```

    }
    maxx[u]=std::max(maxx[u],sum-siz[u]);
    if(maxx[u]<maxx[rt])rt=u;
}

int dist[N],dd[N];
void calcdist(int u,int fa){
    dd[++cntt]=dist[u];
    for(int i=head[u];i;i=e[i].nex){
        int v=e[i].v;
        if(v==fa||vis[v])continue;
        dist[v]=dist[u]+e[i].w;
        calcdist(v,u);
    }
}

std::queue<int>q;
void dfs(int u,int fa){
    vis[u]=1;
    q.push(0);
    tf[0]=1;
    for(int i=head[u];i;i=e[i].nex){
        int v=e[i].v;
        if(v==fa||vis[v])continue;
        dist[v]=e[i].w;
        calcdist(v,u);
        for(int k=1;k<=cntt;k++){
            for(int j=1;j<=m;j++){
                if(p[j]>=dd[k])ret[j]|=tf[p[j]-dd[k]];
            }
        }
        for(int k=1;k<=cntt;k++){
            if(dd[k]<=10000100)q.push(dd[k]),tf[dd[k]]=1;
        }
        cntt=0;
    }
    while(!q.empty())tf[q.front()]=0,q.pop();
    for(int i=head[u];i;i=e[i].nex){
        int v=e[i].v;
        if(v==fa||vis[v])continue;
        sum=siz[v];
        rt=0;
        maxx[rt]=INF;
        calcsiz(v,u);
        calcsiz(rt,-1);
        dfs(rt,-1);
    }
}
}

```

```
int main(){
    scanf("%d%d",&n,&m);
    for(int u,v,w,i=1;i<=n-1;i++){
        scanf("%d%d%d",&u,&v,&w);
        add(u,v,w);
        add(v,u,w);
    }
    for(int i=1;i<=m;i++)scanf("%d",p+i);
    sum=n;
    rt=0;
    maxx[rt]=INF;
    calcsiz(1,-1);
    calcsiz(rt,-1);
    dfs(rt,-1);
    for(int i=1;i<=m;i++){
        if(ret[i])puts("AYE");
        else puts("NAY");
    }
    return 0;
}
```

字符串部分

```
//KMP
void pre(string s){
    int len=s.length();
    pi[0]=0;
    for (int i=1;i<len;i++){
        int j=pi[i-1];
        while(j&& s[i]!=s[j])j=pi[j-1];
        pi[i]=j+(s[i]==s[j]);
    }
}

//ACAM
namespace ACAM {
    struct tire{
        int son[26],fail,in,id,ans;
        void clear(){
            fail=in=id=ans=0;
            memset(son,0,sizeof(son));
        }
    }T[N];
    int cntid,cntn,ans[N];
    queue<int>Q;
    void init(){
        cntid=cntn=0;
        T[0].clear();
    }
    int insert(string s){//return id;
        int u=0,len=s.length();
        for (int i=0;i<len;i++){
            int& son=T[u].son[s[i]-'a'];//地址写法更方便
            if (!son)son=++cntn,T[son].clear();
            u=son;
        }
        if (!T[u].id)T[u].id=++cntid;//防止重复
        return T[u].id;//记录编号
    }
    void build(){
        //建立ac自动机
        while (!Q.empty()) Q.pop();
        for (int i=0;i<26;i++)
            if (T[0].son[i])Q.push(T[0].son[i]);
    }
}
```



```

while (!Q.empty()){
    int u=Q.front();Q.pop();
    for (int i=0;i<26;i++){
        if (T[u].son[i]){
            T[T[u].son[i]].fail=T[T[u].fail].son[i];
            T[T[T[u].fail].son[i]].in++;
            Q.push(T[u].son[i]);
        }
        else T[u].son[i]=T[T[u].fail].son[i];//改变树的结构
    }
}
}

void qry(string t){
    int u=0,len=t.length();
    for (int i=0;i<len;i++){
        u=T[u].son[t[i]-'a'];
        T[u].ans++;
    }
}

void topo(){
    while (!Q.empty()) Q.pop();
    for (int i=0;i<=cntn;i++){
        if (!T[i].in)Q.push(i);
    }
    while (!Q.empty()){
        int u=Q.front();Q.pop();
        ans[T[u].id]=T[u].ans;
        int v=T[u].fail;
        T[v].ans+=T[u].ans;
        if (!--T[v].in)Q.push(v);
    }
}
}
}

```

//后缀排序

```

int n,x[N],y[N],c[N],sa[N],m;
char s[N];
void SA(){
    m=128;
    for(int i=1;i<=n;i++)c[x[i]=s[i]]++;
    for(int i=1;i<=m;i++)c[i]+=c[i-1];
    for(int i=n;i>=1;i--)sa[c[x[i]]--]=i;
    for(int k=1;k<=n;k<=<1){

```

```

        memset(c,0,sizeof(c));
        for(int i=1;i<=n;i++)y[i]=sa[i];
        for(int i=1;i<=n;i++)c[x[y[i]+k]]++;
        for(int i=1;i<=m;i++)c[i]+=c[i-1];
        for(int i=n;i>=1;i--)sa[c[x[y[i]+k]]--]=y[i];
        memset(c,0,sizeof(c));
        for(int i=1;i<=n;i++)y[i]=sa[i];
        for(int i=1;i<=n;i++)c[x[y[i]]]++;
        for(int i=1;i<=m;i++)c[i]+=c[i-1];
        for(int i=n;i>=1;i--)sa[c[x[y[i]]]--]=y[i];
        m=0;
        for(int i=1;i<=n;i++)y[i]=x[i];
        for(int i=1;i<=n;i++){
            if(y[sa[i]]==y[sa[i-1]]&&y[sa[i]+k]==y[sa[i-1]+k])
                x[sa[i]]=m;
            else x[sa[i]]++;
        }
        if(n==m)break;
    }
}

```

//回文自动机

```

int pos,cur,fail[N],cnt=1,num[N],len[N],last,tr[N][26];
char s[N];
int getfail(int x,int i){
    while(i-len[x]-1<0||s[i-len[x]-1]!=s[i])x=fail[x];
    return x;
}
int main(){
    scanf("%s",s);
    len[1]=-1;fail[0]=1;
    for(int i=0;s[i];i++){
        if(i>=1)s[i]=(s[i]+last-97)%26+97;
        pos=getfail(cur,i);
        if(!tr[pos][s[i]-'a']){
            fail[++cnt]=tr[getfail(fail[pos],i)][s[i]-'a'];
            tr[pos][s[i]-'a']=cnt;
            num[cnt]=num[fail[cnt]]+1;
            len[cnt]=len[pos]+2;
        }
        cur=tr[pos][s[i]-'a'];
        last=num[cur];
        printf("%d ",last);
    }
}

```

```

    }
    return 0;
}

//马拉车
#include<bits/stdc++.h>
#define N 15000006
int n,maxright,mid,hw[N*2+250],ans;
char a[N],s[N*2+250];
void manacher(){
    maxright=0;
    for(int i=1;i<n;i++){
        if(i<maxright){
            hw[i]=std::min(hw[(mid<<1)-i],hw[mid]+mid-i);
        }
        else hw[i]=1;
        while(s[i-hw[i]]==s[i+hw[i]])hw[i]++;
        if(i+hw[i]>maxright){
            maxright=i+hw[i];
            mid=i;
        }
    }
}

void init(){
    s[0]=s[1]='^';
    for(int i=0;i<n;i++){
        s[i*2+2]=a[i];
        s[i*2+3]='^';
    }
    n=n*2+2;
    s[n]=0;
}

int main(){
    std::cin>>a;
    n=strlen(a);
    init();
    manacher();
    for(int i=0;i<n;i++){
        ans=std::max(ans,hw[i]);
    }
    printf("%d\n",ans-1);
    return 0;
}

```


数据结构部分

```
for(int j=1;j<=logn;j++)
    for(int i=1;i+(1<<j)-1<=n;i++)
        d[i][j]=max(d[i][j-1],d[i+(1<<(j-1))][j-1]);
for(int i=1;i<=m;i++){
    int x=read(),y=read();
    int s=longn[y-x+1];
    printf("%d\n",max(d[x][s],d[y-(1<<s)+1][s]));
}

//LCT
#define N 300050
#define ls ch[x][0]
#define rs ch[x][1]
#define isroot(x) (ch[f[x]][0]!=x&&ch[f[x]][1]!=x)
#define get(x) (ch[f[x]][1]==x)
int n,m,ch[N][2],f[N],s[N],v[N],r[N],top,st[N];
void pushup(int x){
    s[x]=s[ls]^s[rs]^v[x];
}
void pushr(int x){
    std::swap(ls,rs);
    r[x]^=1;
}
void pushdown(int x){
    if(r[x]){
        if(ls)pushr(ls);
        if(rs)pushr(rs);
        r[x]=0;
    }
}
void rotate(int x){
    int y=f[x],z=f[y],k=get(x);
    if(!isroot(y))ch[z][get(y)]=x;
    ch[y][k]=ch[x][!k],f[ch[x][!k]]=y;
    ch[x][!k]=y,f[y]=x,f[x]=z;
    pushup(y);pushup(x);
}
void splay(int x){
    top=1;st[top]=x;
    for(int i=x;!isroot(i);i=f[i])st[++top]=f[i];
```

```

        for(int i=top;i;i--)pushdown(st[i]);
        for(int fa;fa=f[x],!isroot(x);rotate(x))
            if(!isroot(fa))rotate(get(fa)==get(x)?fa:x);
    }
    void access(int x){
        for(int p=0;x;x=f[p=x])
            splay(x),rs=p,pushup(x);
    }
    void makeroot(int x){
        access(x);splay(x);pushr(x);
    }
    int findroot(int x){
        access(x);
        splay(x);
        while(ls)pushdown(x),x=ls;
        splay(x);
        return x;
    }
    void split(int x,int y){
        makeroot(x);access(y);splay(y);
    }
    void link(int x,int y){
        makeroot(x);
        if(findroot(y)==x)return;
        f[x]=y;
    }
    void cut(int x,int y){
        makeroot(x);
        if(findroot(y)!=x||f[y]!=x||ch[y][0])return;
        f[y]=rs=0;pushup(x);
    }
    //笛卡尔树
    void init(){
        n=read();
        for(int i=1;i<=n;i++){
            a[i]=read();
            while(a[sta[top]]>a[i]&&top) son[i][0]=sta[top--];
            if(sta[top]) son[sta[top]][1]=i;
            sta[++top]=i;
        }
    }
    void solve(){
        for(int i=1;i<=n;i++){

```

```

        ans1=ans1^(111*i*(son[i][0]+1));
        ans2=ans2^(111*i*(son[i][1]+1));
    }

    printf("%lld %lld",ans1,ans2);
}

//splay
#define N 200006

struct node{
    int ch[2];
    int siz,val,tag,fa;
    void init(int v,int f){
        ch[0]=ch[1]=0;
        val=v;fa=f;tag=0;siz=1;
    }
}s[N];
int root,n,m,tot,l,r;
void pushup(int p){
    s[p].siz=s[s[p].ch[0]].siz+s[s[p].ch[1]].siz+1;
}

void pushdown(int p){
    if(s[p].tag){
        s[s[p].ch[0]].tag^=1;
        s[s[p].ch[1]].tag^=1;
        s[p].tag=0;
        std::swap(s[p].ch[0],s[p].ch[1]);
    }
}

void rot(int x){
    int y=s[x].fa,z=s[y].fa,k=s[y].ch[1]==x;
    if(z)
        s[z].ch[s[z].ch[1]==y]=x;
    s[x].fa=z;
    s[y].ch[k]=s[x].ch[k^1];
    s[s[x].ch[k^1]].fa=y;
    s[x].ch[k^1]=y;
    s[y].fa=x;
    pushup(x);pushup(y);
}

void splay(int x,int goal){

```

```

while(s[x].fa!=goal){
    int y=s[x].fa,z=s[y].fa;
    if(z!=goal)
        rot((s[y].ch[1]==x)^(s[z].ch[1]==y)?x:y);
    rot(x);
}
if(goal==0)root=x;
}

```

```

int kth(int k){
    int u=root;
    while(1){
        pushdown(u);
        if(k<=s[s[u].ch[0]].siz)u=s[u].ch[0];
        else if(s[s[u].ch[0]].siz+1==k)return u;
        else k-=s[s[u].ch[0]].siz+1,u=s[u].ch[1];
    }
}

```

```

void insert(int x){
    int u=root,f=0;
    while(u)f=u,u=s[u].ch[x>s[f].val];
    u++tot;
    if(f)
        s[f].ch[x>s[f].val]=u;
    s[u].init(x,f);
    splay(u,0);
}

```

```

void solve(){
    scanf("%d%d",&l,&r);
    l=kth(l);
    r=kth(r+2);
    splay(l,0);
    splay(r,l);
    s[s[s[l].ch[1]].ch[0]].tag^=1;
}

```

```

void write(int x){
    pushdown(x);
    if(s[x].ch[0])write(s[x].ch[0]);
    if(s[x].val>1&& s[x].val<n+2)printf("%d ",s[x].val-1);
    if(s[x].ch[1])write(s[x].ch[1]);
}

```

//可持久化并查集


```

#define N 200050
#define ls t[p].lson
#define rs t[p].rson
#define v t[p].val
#define fa t[p].f
#define mid (l+r>>1)
#define max(x,y) (x>y?x:y)
int n,m,rt[N],tot;
struct segtree{
    int val,f,lson,rson;
}t[N*50];
int build(int l,int r){
    int p=++tot;
    if(l==r){
        v=1;fa=l;return p;
    }
    ls=build(l,mid);
    rs=build(mid+1,r);
    return p;
}
int query_fa(int p,int l,int r,int pos){
    if(l==r)return fa;
    if(pos<=mid)return query_fa(ls,l,mid,pos);
    else return query_fa(rs,mid+1,r,pos);
}
int query_rk(int p,int l,int r,int pos){
    if(l==r)return v;
    if(pos<=mid)return query_rk(ls,l,mid,pos);
    else return query_rk(rs,mid+1,r,pos);
}
int upd_val(int now,int l,int r,int pos,int vv){
    int p=++tot;
    t[p]=t[now];
    if(l==r){
        v=max(v,vv);return p;
    }
    if(pos<=mid)ls=upd_val(t[now].lson,l,mid,pos,vv);
    else rs=upd_val(t[now].rson,mid+1,r,pos,vv);
    return p;
}
int upd_fa(int now,int l,int r,int pos,int f){
    int p=++tot;
    t[p]=t[now];

```

```

        if(l==r){
            fa=f;return p;
        }
        if(pos<=mid)ls=upd_fa(t[now].lson,l,mid,pos,f);
        else rs=upd_fa(t[now].rson,mid+1,r,pos,f);
        return p;
    }
    int find(int p,int pos){
        int f=query_fa(p,1,n,pos);
        if(f==pos)return f;
        else return find(p,f);
    }
    void init(){
        scanf("%d%d",&n,&m);
        rt[0]=build(1,n);
    }
    void solve(){
        int o,a,b;
        for(int i=1;i<=m;i++){
            scanf("%d",&o);
            if(o==1){
                scanf("%d%d",&a,&b);
                int x=find(rt[i-1],a),y=find(rt[i-1],b);
                if(x==y)rt[i]=rt[i-1];
                else{
                    int sx=query_rk(rt[i-1],1,n,x),sy=query_rk(rt[i-1],1,n,y);
                    if(sx>sy)std::swap(sx,sy),std::swap(x,y);
                    int tmp=upd_fa(rt[i-1],1,n,x,y);
                    rt[i]=upd_val(tmp,1,n,y,sx+1);
                }
            }
            else if(o==2){
                scanf("%d",&a);
                rt[i]=rt[a];
            }
            else {
                scanf("%d%d",&a,&b);rt[i]=rt[i-1];
                printf("%d\n",find(rt[i],a)==find(rt[i],b));
            }
        }
    }
}

```

//线段树分裂

```
int n,m,cnt,cnt1,ct;
int bac[N*50],ls[N*50],rs[N*50],rt[N];
ll tr[N*50];
void Del(int y){
    bac[++cnt1]=y;
    ls[y]=rs[y]=tr[y]=0;
}
int New(){
    return cnt1?bac[cnt1--]:++cnt;
}
void pushup(int p){
    tr[p]=tr[ls[p]]+tr[rs[p]];
}
int change(int p,int l,int r,int v,int k){
    if(!p)p=New();
    if(l==r){
        tr[p]+=k;//l?
        return p;
    }
    if(v<=mid)ls[p]=change(ls[p],l,mid,v,k);
    else rs[p]=change(rs[p],mid+1,r,v,k);
    pushup(p);
    return p;
}
int split(int x,int y,ll k){
    if(!x)return x;/**
    ll v=tr[ls[x]];
    y=New();
    if(v<k)rs[y]=split(rs[x],rs[y],k-v);
    else std::swap(rs[x],rs[y]);
    if(v>k)ls[y]=split(ls[x],ls[y],k);//v?
    tr[y]=tr[x]-k;
    tr[x]=k;
    return y;
}
int merge(int u,int v,int l,int r){
    if(!u||!v)return u|v;
    if(l==r){
        tr[u]+=tr[v];return u;
    }
    ls[u]=merge(ls[u],ls[v],l,mid);
    rs[u]=merge(rs[u],rs[v],mid+1,r);
```

```

        pushup(u);
        Del(v);
        return u;
    }
    int kth(int p,int l,int r,int k){
        int v=tr[ls[p]];
        if(l==r){
            return l;
        }
        if(k<=v)return kth(ls[p],l,mid,k);
        else return kth(rs[p],mid+1,r,k-v);
    }
    ll qry(int p,int l,int r,int x,int y){
        if(l>y||r<x)return 1ll*0;
        if(x<=l&&r<=y){
            return 1ll*tr[p];
        }
        return 1ll*qry(ls[p],l,mid,x,y)+1ll*qry(rs[p],mid+1,r,x,y);
    }

//李超线段树
int n,m,cnt,cnt1,ct;
ll last;
struct line{
    db k,b;
}l[N];
struct node{
    db res;
    int id;
}a[N];
int tr[N<<2];
void make_line(int x,int y,int xx,int yy){
    cnt++;
    if(x==xx){
        l[cnt].k=0;l[cnt].b=std::max(y,yy);
    }
    else {
        l[cnt].k=1.0*(yy-y)/(xx-x);l[cnt].b=1.0*(yy-xx*l[cnt].k);
    }
}

int cmp(db x,db y){
    if(x-y>eps)return 1;
    if(y-x>eps)return -1;

```

```

        else return 0;
    }
    double cale(int id,int x){
        return l[id].b+l[id].k*x;
    }
#define get cale
void upd(int p,int l,int r,int u){
    int&v=tr[p];
    if(cmp(cale(u,mid),cale(v,mid))==1)std::swap(u,v);
    int bl=cmp(cale(u,l),cale(v,l)),br=cmp(cale(u,r),cale(v,r));
    if(bl==1||(!bl&&u<v))upd(ls,l,mid,u);
    if(br==1||(!br&&u<v))upd(rs,mid+1,r,u);
}
void update(int p,int l,int r,int x,int y,int k){
    if(l>=x&&r<=y){
        upd(p,l,r,k);
        return;
    }
    if(x<=mid)update(ls,l,mid,x,y,k);
    if(y>mid)update(rs,mid+1,r,x,y,k);
}
node cmax(node a,node b){
    if(cmp(a.res,b.res)==1)return a;
    if(cmp(a.res,b.res)==-1)return b;
    return a.id<b.id?a:b;
}
node qry(int p,int l,int r,int k){
    if(k<l||k>r)return (node){0,0};
    node a={get(tr[p],k),tr[p]};
    if(l==r)return a;
    return cmax(a,cmax(qry(ls,l,mid,k),qry(rs,mid+1,r,k)));
}
//可持久化线段树

int tr[N*50],ls[N*50],rs[N*50];
int n,m,rt[N],cnt=0,a[N],b[N],tot;
int get(int x){
    return std::lower_bound(a+1,a+tot+1,x)-a;
}
int change(int l,int r,int k,int last){
    int p=++cnt;
    tr[p]=tr[last]+1;ls[p]=ls[last];rs[p]=rs[last];
    if(l==r){return p;}

```

```

        if(k<=mid)ls[p]=change(l,mid,k,ls[p]);
        else rs[p]=change(mid+1,r,k,rs[p]);
        return p;
    }
    int qry(int u,int v,int l,int r,int k){
        if(l==r)return l;
        int x=tr[ls[v]]-tr[ls[u]];
        if(k<=x)return qry(ls[u],ls[v],l,mid,k);
        else return qry(rs[u],rs[v],mid+1,r,k-x);
    }
    //扫描线
    struct segtree{
        int l,r,sum;
        ll len;
    }s[N];
    struct lll{
        int x1,x2,h,mark;
        bool operator<(const lll&x)const{
            return h<x.h;
        }
    }line[N];
    int X[N],n,x1,x2,y,yy,tot;
    void build(int x,int l,int r){
        s[x].l=l;s[x].r=r;
        if(l==r){
            return;
        }
        int mid=l+r>>1;
        build(ls,l,mid);
        build(rs,mid+1,r);
        return;
    }
    void pushup(int x){
        int l=s[x].l,r=s[x].r;
        if(s[x].sum){
            s[x].len=X[r+1]-X[l];
        }
        else s[x].len=s[ls].len+s[rs].len;
        return;
    }
    void update(int x,int L,int R,int mark){
        int l=s[x].l,r=s[x].r;
        if(X[r+1]<=L||X[l]>=R){

```

```

        return;
    }
    if(X[r+1]<=R&&X[l]>=L){
        s[x].sum+=mark;
        pushup(x);
        return;
    }
    update(ls,L,R,mark);
    update(rs,L,R,mark);
    pushup(x);
    return;
}

void init(){
    scanf("%d",&n);
    for(int i=1;i<=n;i++){
        scanf("%d%d%d%d",&x1,&y,&x2,&yy);
        X[i*2-1]=x1,X[i*2]=x2;
        line[i*2-1]=(lll){x1,x2,y,1};
        line[i*2]=(lll){x1,x2,yy,-1};
    }
    n<<=1;
    std::sort(X+1,X+n+1);
    std::sort(line+1,line+n+1);
    tot=std::unique(X+1,X+n+1)-X-1;
    build(1,1,tot-1);
}

void solve(){
    ll ans=0;
    for(int i=1;i<n;i++){
        update(1,line[i].x1,line[i].x2,line[i].mark);
        ans+=s[1].len*(line[i+1].h-line[i].h);
    }
    printf("%lld\n",ans);
}

//线段树合并
void pushup(int p){
    if(tr[ls[p]]>=tr[rs[p]])tr[p]=tr[ls[p]],ans[p]=ans[ls[p]];
    else tr[p]=tr[rs[p]],ans[p]=ans[rs[p]];
}

int change(int root,int l,int r,int k,int x){
    if(!root)root=++tot;
    if(l==r){

```

```

        tr[root]+=x;
        ans[root]=1;
        return root;
    }
    if(k<=mid)ls[root]=change(ls[root],l,mid,k,x);
    else rs[root]=change(rs[root],mid+1,r,k,x);
    pushup(root);
    return root;
}

int merge(int u,int v,int l,int r){
    if(!u||!v)return u|v;
    if(l==r){
        tr[u]+=tr[v];
        ans[u]=1;
        return u;
    }
    ls[u]=merge(ls[u],ls[v],l,mid);
    rs[u]=merge(rs[u],rs[v],mid+1,r);
    pushup(u);
    return u;
}

```


STL部分

- `insert(x)` 当容器中没有等价元素的时候，将元素 `x` 插入到 `set` 中。
- `erase(x)` 删除值为 `x` 的 **所有** 元素，返回删除元素的个数。
- `erase(pos)` 删除迭代器为 `pos` 的元素，要求迭代器必须合法。
- `erase(first,last)` 删除迭代器在 `[first,last)` 的元素。
- `clear()` 清空 `set`。

`insert` 函数的返回值

`insert` 函数的返回值类型为 `pair<iterator, bool>`，其中 `iterator` 是一个指向所插入元素（或者是指向等于

迭代器[](<http://oi-wiki.com/lang/csl/associative-container/#%E8%BF%AD%E4%BB%A3%E5%99%A8> "Per

`set` 提供了以下几种迭代器：

1. `begin()/cbegin()`
返回指向首元素的迭代器，其中 `*begin = front`。
2. `end()/cend()`
返回指向数组尾端占位符的迭代器，注意是没有元素的。
3. `rbegin()/crbegin()`
返回指向逆向数组的首元素的逆向迭代器，可以理解为正向容器的末元素。
4. `rend()/crend()`
返回指向逆向数组末元素后一位置的迭代器，对应容器首的前一个位置，没有元素。

以上列出的迭代器中，含有字符 `c` 的为只读迭代器，你 cannot 通过只读迭代器去修改 `set` 中的元素的值。如果一

查找操作[](<http://oi-wiki.com/lang/csl/associative-container/#%E6%9F%A5%E6%89%BE%E6%93%8D%E4>

- `count(x)` 返回 `set` 内键为 `x` 的元素数量。
- `find(x)` 在 `set` 内存在键为 `x` 的元素时会返回该元素的迭代器，否则返回 `end()`。
- `lower_bound(x)` 返回指向首个不小于给定键的元素的迭代器。如果不存在这样的元素，返回 `end()`。
- `upper_bound(x)` 返回指向首个大于给定键的元素的迭代器。如果不存在这样的元素，返回 `end()`。
- `empty()` 返回容器是否为空。
- `size()` 返回容器内元素个数。

```
| | |
|---|---|
```

`set<int> available;` // 需要大于等于的值 `int x;` // 查找最小的大于等于`x`的元素 `s`
`find(str,pos)` 函数可以用来查找字符串中一个字符/字符串在 `pos`（含）之后第一次出现的位置（若不传参经
`substr(pos, len)` 函数的参数返回从 `pos` 位置开始截取最多 `len` 个字符组成的字符串（如果从 `pos` 于
`insert(index,count,ch)` 和 `insert(index,str)` 是比较常见的插入函数。它们分别表示在 `index` 处连

`erase(index, count)` 函数将字符串 `index` 位置开始（含）的 `count` 个字符删除（若不传参给 `count` 则删除从 `index` 位置开始的所有字符）。

`replace(pos, count, str)` 和 `replace(first, last, str)` 是比较常见的替换函数。它们分别表示将从 `pos` 位置开始（含）的 `count` 个字符替换为 `str`，以及将 `first` 到 `last` 之间的所有字符替换为 `str`。